GRAFANA NOTES

1. What is Grafana?

- Grafana is an open-source analytics and interactive visualization tool.
- It helps us visualize, explore and monitor metrics, logs and other data sources through beautiful dashboards.

Key Points *

→ It does NOT store data.
→ It connects to data sources and visualizes data.
→ Supports many data sources like Prometheus, Loki, Elasticsearch, MySQL etc.

2. Why do we use Grafana?

- To visualize metrics and logs in real-time.
- To create powerful and customizable dashboards.
- To monitor application & infrastructure health.
- To set up alerts and get notified.
- To understand system behavior through charts & graphs.

3. Monitoring vs Observability

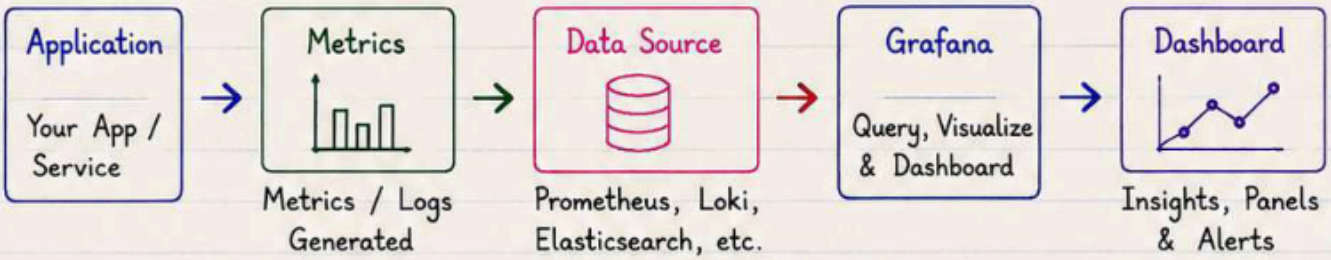
Monitoring

- Tells us WHAT is happening.
- Focus on known issues.
- Uses predefined metrics & alerts.
- Example: CPU usage is high.

Observability

- Helps us understand WHY it is happening.
- Focus on unknown issues.
- Uses metrics, logs, traces & events.
- Example: Why is the CPU usage high?

4. Grafana's Role in a Production System

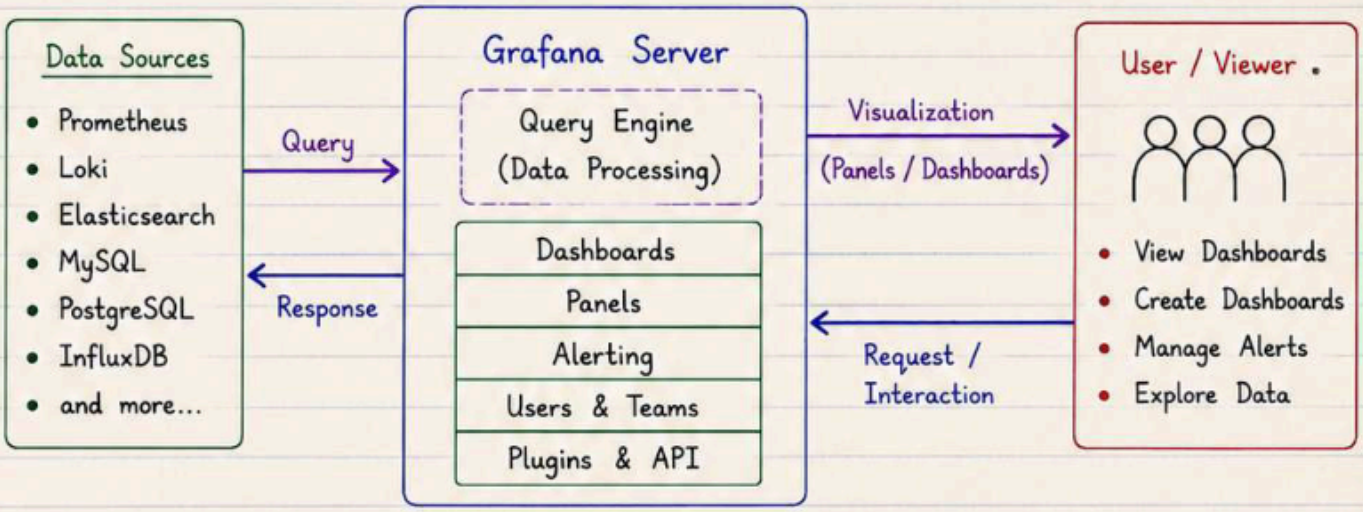


In short: Grafana connects to a data source, queries the data, and displays it in the form of graphs, tables & dashboards.

GRAFANA NOTES

2. Grafana Architecture

Grafana follows a simple and powerful architecture that helps in collecting, querying and visualizing data.



✳ Key Components

(1)	Grafana Server	:	Core of Grafana. Handles requests, processes queries, manages users, dashboards, alerts & plugins.
(2)	Data Sources	:	External systems where data is stored. Grafana queries data from these sources.
(3)	Query Engine	:	Executes queries on data sources and returns the results to be visualized.
(4)	Dashboards	:	Collection of panels arranged in a meaningful way to monitor and analyze data.
(5)	Panels	:	Individual visualization (graph, chart, table, etc.) that displays data from queries.
(6)	Alerting	:	Define alert rules, conditions and notifications based on metric values.
(7)	Users & Teams	:	Manage access, permissions and collaborate on dashboards.
(8)	Plugins & API	:	Extend Grafana functionality using plugins and integrate via API.

Flow: Data Source → Query Engine → Grafana Server → Dashboard / Panels → User

GRAFANA NOTES

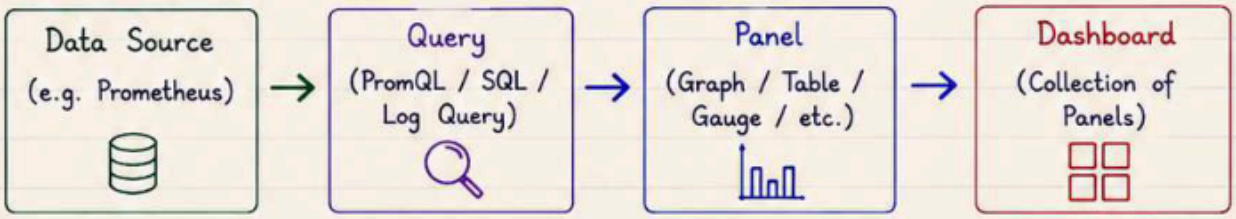
3. Data Sources

- Grafana gets data from external systems called Data Sources.
- Grafana does not store data (except some internal metadata). It only queries and visualizes it.

Popular Data Sources

Prometheus *	Loki	Elasticsearch	InfluxDB	MySQL / PostgreSQL	Others
• Most popular • Time series metrics • Pull based • Used for infra & app metrics	• Log aggregation • Designed by Grafana Labs • Works like Prometheus but for logs	• Search & analytics engine • Stores logs & events • Powerful querying	• Time series database • High write performance • Good for IoT & metrics	• Relational databases • Use SQL queries • Good for business data	• CloudWatch • Azure Monitor • Google Cloud Monitoring • MongoDB and more...

How it Works ?



Types of Data in Grafana

- Metrics (Time Series) : Numeric values over time. (e.g. CPU usage, Request count)
- Logs : Unstructured / semi-structured text events.
- Traces : Request flow across services (with Tempo / Jaeger).

* Grafana + Prometheus is the most widely used combination in monitoring systems.

Important Points

- ☑ Grafana is a visualization layer, not a storage.
- ☑ The power of Grafana comes from the data source behind it.

Example Flow: Application Metrics → Prometheus → Grafana → Dashboard

GRAFANA NOTES

4. Dashboards & Panels

- Dashboard : Collection of panels arranged together.
- Panel : Visualization of a single query.
- Grafana provides many visualization types to understand your data better.

Key Points *

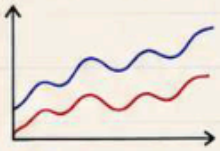
→ Dashboards help in monitoring complete system at a glance.
→ Each panel is based on a query from a data source.
→ You can customize time range, refresh interval, filters etc.

Components of a Dashboard

Time Range	:	Select time period (Last 5m, 1h, 1d, 7d, custom)
Refresh Interval	:	Auto refresh data (5s, 10s, 1m, 5m, etc.)
Variables	:	Reusable values in dashboard (e.g. $application, $env)
Filters	:	Narrow down data based on labels or values
Annotations	:	Add notes or events on the graph

Common Panel Types

Time Series



Best for metric trends over time.

Stat

125
Requests / sec

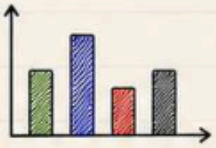
Displays single stat value.

Table

Time	Value	Status
10:00	120	OK
10:01	135	OK
10:02	98	WARN

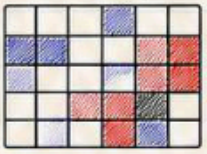
Shows data in tabular format.

Bar Chart



Compare values across categories.

Heatmap



Show intensity in 2D.

Pie Chart



Show percentage distribution.

Logs Panel

10:00:01 INFO User logged in
10:00:02 ERROR DB timeout
10:00:03 INFO Request ok
10:00:04 WARN High latency

View logs from Loki or other sources.

Histogram



Show distribution of values.

Stat + Sparkline

98



Value with mini trend.

Text

Server Healthy

Display text or status info.

Dashboard Best Practices

- Keep dashboards simple and focused.
- Use meaningful names and consistent units.
- Avoid too many panels in a single dashboard.
- Use variables & filters for flexibility.
- Group related metrics together.

Example Use Cases

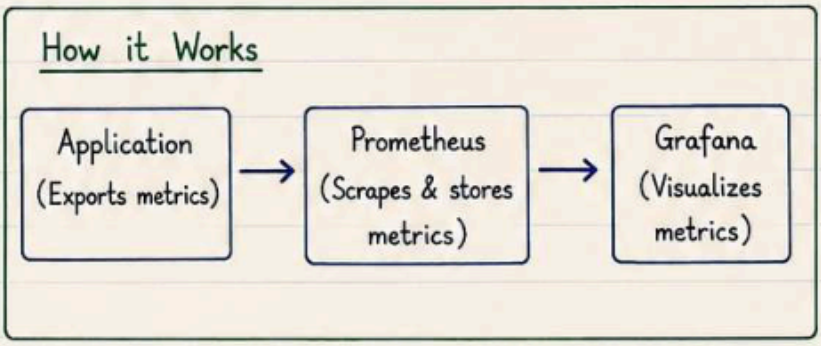
✓ Application performance monitoring
✓ Infrastructure & server monitoring
✓ Business metrics tracking
✓ Error rate & alert monitoring

GRAFANA NOTES

5. Prometheus + Grafana

Prometheus is the most popular data source used with Grafana for metrics monitoring.

How it Works



Key Points

- Prometheus collects metrics in real-time.
- Grafana queries Prometheus and displays data in dashboards.
- NoSQL database (time series DB).

Prometheus Basics

- Metrics : Collected data (e.g. http_requests_total)
- Labels : Key-Value pairs to identify metrics (e.g. method="GET", status="200")
- Time Series : Metric + Labels + Timestamp
- Job : Group of targets (e.g. spring-boot-app)
- Target : Endpoint from which metrics are scraped (e.g. http://localhost:8080/actuator/prometheus)

Connecting Prometheus to Grafana

① Add Data Source (Prometheus) → ② Enter URL (e.g. http://prometheus:9090) → ③ Save & Test → ④ Start Building Dashboards

PromQL (Prometheus Query Language)

Used to query metrics from Prometheus.

Query	Description	Example
Instant Query	Get value at a specific point in time	up
Range Query	Get values over a time range	up[5m]
Rate	Per second rate of increase	rate(http_requests_total[5m])
Sum	Sum of values	sum(http_requests_total)
Avg	Average of values	avg(cpu_usage_percent)

Example Use Case

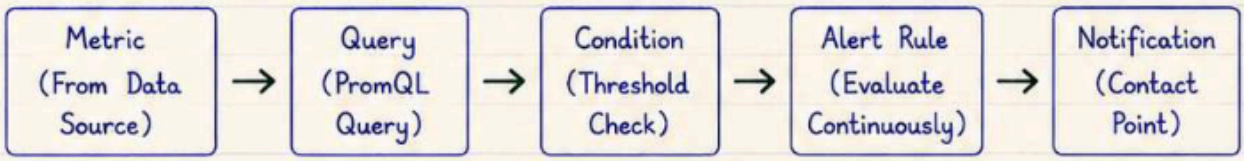
You want to monitor request rate of your API over last 5 minutes.
Query : rate(http_requests_total[5m])
This will show how many requests are coming per second.

GRAFANA NOTES

6. Grafana Alerting

Alerting in Grafana helps you get notified when something goes wrong in your system.

Alerting Flow



Key Concepts

1. **Alert Rule** : Defines what to monitor and the condition.
2. **Query** : PromQL query that returns the metric.
3. **Condition** : When the metric crosses the threshold. (e.g. >, <, >=, <=, ==)
4. **Evaluation** : How often the rule is evaluated (e.g. every 30s).
5. **For** : Condition must be true for a specific time before firing alert (to avoid flapping).
6. **Contact Point** : Where the alert will be sent (Email, Slack, Webhook).
7. **Notification** : Message sent when alert is triggered or resolved.

Example Alert Rule

Name	:	High CPU Usage
Query	:	`avg(rate(node_cpu_seconds_total{mode="idle"}[5m]))`
Condition	:	WHEN avg() < 20
For	:	5m
Evaluate Every	:	30s
Contact Point	:	Slack Channel
Message	:	CPU usage is high on {{ $labels.instance }}

Notification Channels

- Email
- Slack
- Webhook
- PagerDuty
- Microsoft Teams

Best Practices

- Don't alert on single data points.
- Use 'For' clause to avoid false alerts.
- Keep alert names meaningful.
- Monitor important business metrics, not everything.
- Test alerts regularly.

GRAFANA NOTES

7. Annotations in Grafana

Annotations help you mark important events on your dashboards so you can correlate metrics with events.

What are Annotations?

> Annotations are markers on graphs that show significant events, deployments, incidents, or changes over time.

How it Works

Create Annotation Source (e.g. Prometheus, Loki) → Grafana Fetches Annotation Data → Annotations are Displayed on Dashboard Panels → Helps in Identifying Events Quickly

Types of Annotations

- Query Annotations : Fetched from data sources (Prometheus, Loki, Elasticsearch).
- Dashboard Annotations : Manually added annotations for specific dashboards.
- API Annotations : Added using Grafana HTTP API.
- Alert Annotations : Automatically created when alerts are triggered.

Add Annotation Manually

① Go to Dashboard → ② Click on Panel → ③ Click on Annotations → ④ Add Annotation

Example Annotation

Name	:	Deployment
Type	:	Event
Time	:	2025-08-07 14:30
Text	:	Deployed v1.2.0 to Production
Tags	:	deployment, production
Panel	:	CPU Usage

Common Use Cases

- Mark deployments or releases
- Note incidents or outages
- Track configuration changes
- Correlate alerts with events

Best Practices

- Use annotations to give context to metrics.
- Keep annotation names meaningful.
- Use consistent tagging for easy filtering.
- Don't overload dashboards with too many annotations.
- Regularly review and clean up old annotations.

GRAFANA NOTES

8. Variables in Grafana

Variables make dashboards dynamic and reusable.
They allow users to select values (like server, service, region) and update all panels at once.

Key Points *

- Variables provide dropdown filters for dashboards.
- They are defined at dashboard level.
- When value changes, all related panels update.

How it Works

Create Variable (Name, Type, Query) → Select Value (Dropdown on Dashboard) → Value is Used in Queries of Panels → Panels Update Automatically

Types of Variables

- Query Variable : Values are fetched using a query from data source.
- Custom Variable : Manually defined list of values.
- Constant Variable : Single fixed value.
- Interval Variable : Predefined time intervals (e.g. 1m, 5m, 1h).
- Data Source Variable : Used to switch between data sources.

Create a Query Variable (Example)

Step 1 : Go to Dashboard Settings → Variables → Add Variable
Step 2 : Configure Variable

Name	: server
Type	: Query
Data Source	: Prometheus
Query	: label_values(up, instance)
Refresh	: On Dashboard Load
Multi-value	: Enabled (optional)
Include All	: Enabled (All value = .*)

How it Looks

Server: All ▼
Options:
- All
- server-1
- server-2
- server-3

(selected value updates panels)

Use Variable in Panel Query (PromQL Example)

Without Variable : `up{instance="server-1"}`
With Variable : `up{instance="$server"}`

Note
Use $variable_name in queries to reference the selected value.

Best Practices

- Use meaningful variable names.
- Limit number of variables to avoid confusion.
- Use "All" option carefully (can increase query load).
- Set default value for better user experience.
- Test variables to ensure panels update correctly.